NLP Tasks, Text and Counting Models

Task Map, Preprocessing, Arabic, Text Classifiers and N-gram Language Models

Naeemullah Khan
naeemullah.khan@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

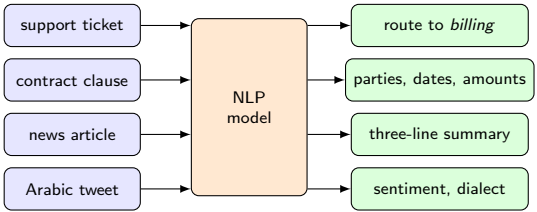
November 8, 2026

Table of Contents

1. Motivation: text as the main carrier of knowledge, ambiguity, and four eras of NLP
2. The NLP task map: classification, sequence labeling, question answering, summarization, translation, and text-to-text
3. Scoring each task: accuracy and macro-F1, entity F1, exact match, ROUGE, BLEU, and chrF
4. The classic preprocessing pipeline: cleaning, normalization, tokenization, stop words, stemming and lemmatization
5. What makes Arabic hard: script, root-and-pattern morphology, clitics, diacritics, and dialects
6. Bag of words and TF-IDF: vocabulary, document-term matrix and sparsity; n-gram features and reweighting
7. Text classifiers from counts: Naive Bayes worked by hand, then logistic regression, and the baseline to beat
8. N-gram language models: from labels to next words; the chain rule, the Markov assumption, counting n-grams, start and end tokens, sampling, <UNK> and sparsity
9. Summary and references

Most Knowledge Arrives as Text

- Policies, contracts, manuals, emails, tickets, reports and chats: an organisation records what it knows in written language.
- People read it slowly and one document at a time. A program can read millions, but only once the text has been turned into numbers.
- **Natural language processing (NLP)** is the set of methods that map text to something a program can act on: a label, a list of entities, an answer, a summary or a translation.
- The same few input-output shapes cover almost every application, from spam filters to chat assistants.



Unstructured text goes in; a structured output that software can use comes out.

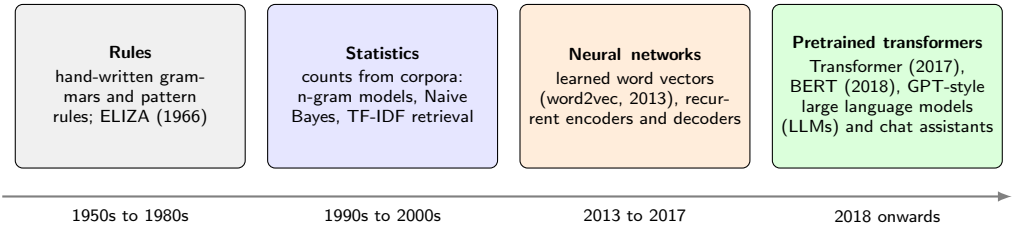
Why Language Is Hard for Machines

The same string can carry several meanings, and the same meaning can be written many ways.

Problem	Example	What resolves it
Word sense	"She sat by the *bank*" (river or money?)	surrounding words
Attachment	"I saw the man *with the telescope*" (who holds it?)	world knowledge
Reference	"Sara told Huda that *she* had won" (who won?)	earlier context
Missing vowels	علم reads as *science*, *flag*, *taught* or *knew*	sentence context
Variation	"cheap flight" and "low-cost airfare"; one question in five Arabic dialects	a notion of similarity

- Every method in this course is, in the end, a way of using context to pick the intended reading and of measuring similarity beyond exact string match.

Four Eras of NLP



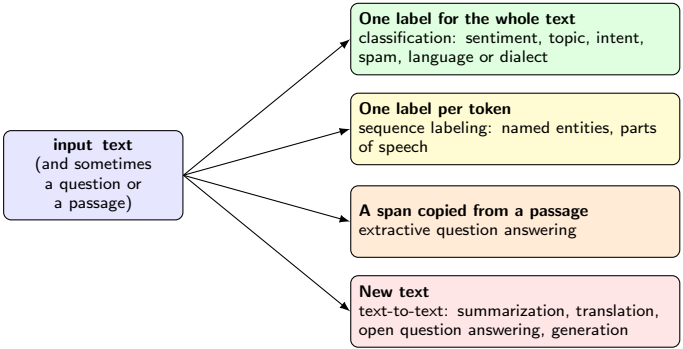
- Each era kept something useful. TF-IDF is still the first baseline for classification and the keyword half of modern search; tokenization and word vectors feed every large language model.
- Today starts at the statistical era: the tasks, the pipeline that prepares text, and the bag-of-words baseline.

Learning Outcomes

By the end of this lecture you should be able to:

- **Map** an NLP problem to its task type (classification, sequence labeling, extractive question answering or text-to-text) and **choose** the metric that scores it.
- **Build** a classic preprocessing pipeline and **decide** which steps help or hurt a given task.
- **Explain** what makes Arabic hard to process: the script, root-and-pattern morphology, attached clitics, missing diacritics and dialects.
- **Build** a document-term matrix by hand, **explain** what bag of words loses, and **compute** TF-IDF.
- **Train** a Naive Bayes classifier by counting, **classify** a review with log-likelihood ratios, and **compare** it with logistic regression.
- **Estimate** n-gram probabilities from a corpus, **score** a sentence with start, end and `<UNK>` tokens, and **explain** why counting models are sparse.
- **Prerequisite knowledge:** Python; supervised learning with train, validation and test splits; logistic regression; precision, recall and F1; conditional probability; neural networks trained by gradient descent in PyTorch.

Every NLP Task Maps Text to an Output



Why the shape matters

The output shape fixes the model head, the loss and the metric. Name the shape first; the method follows.

Text Classification

- One label (or a set of labels) for a whole document, sentence or message.

Task	Input	Output
Sentiment	"The battery died after two days."	negative
Topic	a news article about a football final	sports
Intent	أريد حجز رحلة إلى جدة "I want to book a flight to Jeddah"	`book_flight`
Spam	"You have won a prize, click here"	spam
Dialect identification	an Arabic sentence	Gulf, Egyptian, Levantine, ...

- **Multi-class** picks one of K labels; **multi-label** may assign several (a ticket can be both *billing* and *urgent*).
- Real label distributions are skewed: most tickets are routine, few are urgent. A model that always predicts the majority class can look accurate and be useless, which is why the metric matters.

Sequence Labeling: Named Entity Recognition

- One decision per token. **Named entity recognition (NER)** marks names of people, organisations, places and dates.
- **BIO tags** turn spans into per-token labels: **B** begins an entity, **I** continues it, **O** is outside any entity.

Token	Fatimah	joined	Saudi	Aramco	in	Dhahran	in	2019
Tag	B-PER	O	B-ORG	I-ORG	O	B-LOC	O	B-DATE

- The two-token organisation shows why I is needed: without it, “Saudi” and “Aramco” would be read as two separate entities.
- Part-of-speech tagging has the same shape: one grammatical tag (noun, verb, . . .) per token.
- English NER gets a strong cue from capital letters. Arabic script has no capitals, so an Arabic tagger must rely on context and word lists alone.

Question Answering

- **Extractive question answering (QA):** given a question and a passage, return the start and end of the answer span inside the passage. The model can only copy, so it cannot invent an answer.
- SQuAD, the dataset that popularised it, has 100,000+ crowdworker questions on Wikipedia articles. In 2016 a logistic regression baseline reached 51.0 F1 against 86.8 for humans.
- **Generative QA:** the model writes the answer. *Open-book* when a passage is supplied, *closed-book* when it answers from what it memorised in training, which is where hallucinated answers come from.

Passage: King Abdullah University of Science and Technology opened in **2009** in Thuwal, on the Red Sea coast north of Jeddah.

Question: When did KAUST open?

Extractive answer: the span "2009" (characters 61 to 64, counting from 0)

Generative answer: "KAUST opened in 2009."

Same question, two output shapes: a pointer into the text, or new text.

Rajpurkar et al., "SQuAD: 100,000+ Questions for Machine Comprehension of Text," EMNLP 2016, arXiv:1606.05250v3.

Summarization and Machine Translation

Summarization

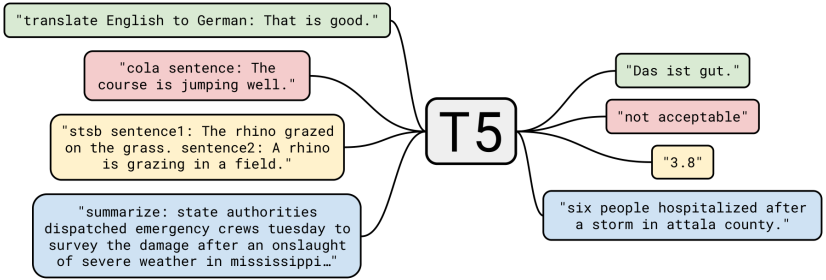
- **Extractive:** select the most important sentences and copy them. Faithful by construction, but often choppy.
- **Abstractive:** write new sentences that compress the content. Fluent, but it can state things the source never said.
- Length and focus are part of the task: a summary for a manager and one for an engineer differ.

Machine translation

- Map a sentence in one language to the same meaning in another, for example English to Arabic.
- Word order changes (Arabic often puts the verb first), one word can become several, and gender or number must agree in the target language.
- Like summarization, there are many correct outputs for one input.

Consequence: both are *text-to-text* tasks with no single right answer, so scoring them is far harder than checking a label.

One Interface for Every Task: Text-to-Text



Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," JMLR 2020, arXiv:1910.10683v4, Figure 1.

- Translation, acceptability judgments, sentence similarity (the "3.8") and summarization all become "text in, text out", with the task named by a prefix in the input.
- One model, one loss and one decoding procedure then serve every task. Prompting a chat assistant uses the same idea, with the prefix replaced by a written instruction.

How Each Task Is Scored

Task	Metric	What it rewards
Classification	accuracy; **macro-F1**	accuracy counts correct predictions; macro-F1 averages F1 over classes, so a rare class counts as much as a common one
NER	entity-level F1	a predicted entity is correct only if both its span and its type match the reference
Extractive QA	exact match (EM); token F1	EM: the span equals the reference; token F1: partial credit for word overlap
Summarization	ROUGE	n-gram overlap with a reference summary, recall-oriented (how much of the reference is covered)
Translation	BLEU; chrF	BLEU: n-gram precision against references, with a brevity penalty for short outputs; chrF: F-score over *character* n-grams
Open generation	human or LLM judges	quality criteria that no reference string can capture

- A task is fully defined only once its score is fixed: the same model can look good or bad depending on the metric. An **n-gram** is a run of n consecutive words (or characters): "the cat" is a word bigram.
- chrF gives partial credit when two forms share most of their characters, as inflected forms of one stem do. At WMT14 (the 2014 Workshop on Statistical Machine Translation), its variant chrF3 correlated best with human rankings, sentence by sentence, for translation out of English.

Papineni et al., "BLEU," ACL 2002, P02-1040; Lin, "ROUGE," 2004, W04-1013; Popović, "chrF," WMT 2015, W15-3049.

Reference-Based Metrics, Worked by Hand

Reference: "the cat sat on the mat" (6 words, 5 bigrams)

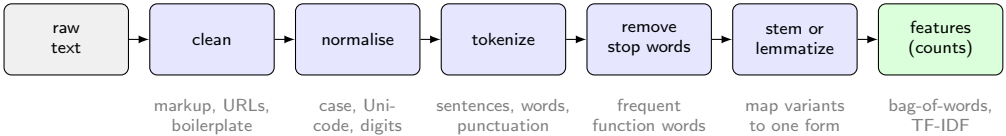
Candidate	Unigram precision	Bigram precision
the cat *is* on the mat	5/6	3/5
a feline rested on the rug	2/6	1/5

- Count each candidate n-gram that also appears in the reference, **clipped** to how often it appears there ("the" counts at most twice).
- Divide by the candidate's n-gram count for precision (BLEU), or by the reference's for recall (ROUGE). Here both lengths are equal, so precision and recall coincide.

The failure case. The second candidate is a correct paraphrase, yet it scores far lower than the first, which changed the meaning of one word.

N-gram metrics measure *surface overlap*. They are cheap and reproducible for comparing two translation systems on the same test set, and unreliable for judging one free-form answer on its own.

The Classic Preprocessing Pipeline



- Every NLP task starts from raw text, and every model needs numbers. The pipeline is the path between the two: clean the text, cut it into tokens, then count them.
- Each step shrinks the vocabulary so that a count-based model sees fewer, more frequent features.
- Each step also throws information away. Whether that helps depends on the task, so every step is a choice to test, never a default to apply.

Cleaning and Normalisation

- **Cleaning** removes what is not language: HTML tags, URLs, signatures, repeated headers.
- **Case folding** maps “Great” and “great” to one feature, at the cost of the capitals that mark names.
- **Unicode NFKC normalisation** folds characters that look alike into one code point: the ligature “*fi*” into two letters, full-width digits into ASCII digits, and Arabic presentation forms into ordinary Arabic letters.
- NFKC does **not** convert Arabic-Indic digits (the digit shapes used in Arabic text) to 0 to 9; that mapping has to be added by hand if the task needs it.

Before:
`Check out https://example.com/deal!!!  The <b>BEST</b> fi nancial app of 2026 :)`

After removing tags and URLs, NFKC, lower case, collapsed spaces:
`check out the best financial app of 2026 :)`

In the input, “*fi*” is the single ligature character U+FB01 and 2026 is typed in full-width digits; NFKC turns both into plain characters. Output of Python’s `unicodedata.normalize` with two regular expressions. The emoticon survives, and for sentiment it should.

Tokenization by Rules

```
from nltk.tokenize import (word_tokenize,
                           sent_tokenize)

word_tokenize("Don't pay $3.50 for e-mail "
              "access in the U.S.A., "
              "it's free!")
# ['Do', "n't", 'pay', '$', '3.50',
#  'for', 'e-mail', 'access', 'in',
#  'the', 'U.S.A.', ',', 'it', "'s",
#  'free', '!']

sent_tokenize("Dr. Smith arrived at 3 p.m. "
              "on Monday. The meeting "
              "started late.")
# ['Dr. Smith arrived at 3 p.m. on Monday.',
#  'The meeting started late.']
```

Outputs from NLTK 3.9.1, run for this slide.

- Splitting on spaces would keep "free!" and "U.S.A.," as single tokens, punctuation attached.
- A rule-based tokenizer separates punctuation, keeps numbers and abbreviations whole, and splits contractions ("Do" + "n't").
- The sentence splitter must know that the full stops in "Dr." and "p.m." do not end a sentence.
- Rules are written per language. For Arabic, splitting attached clitics off the stem is part of tokenization.

Stop Words

- **Stop words** are frequent function words ("the", "is", "of") that appear in nearly every document and say little about its topic.
- Dropping them shrinks the index and helps topic classification and keyword search.
- Standard lists are generic. NLTK's English list has 198 entries and includes *not*, *no*, *don't* and *who*; its Arabic list has 754.
- For sentiment, negation is the signal. For question answering, *who* and *when* are the question.

Input: `the movie was not good at all`

After NLTK's English stop list: `['movie', 'good']`

A negative review now reads as positive.

Remove stop words only when the task is about topic, and check the list against the task first.

Stemming versus Lemmatization

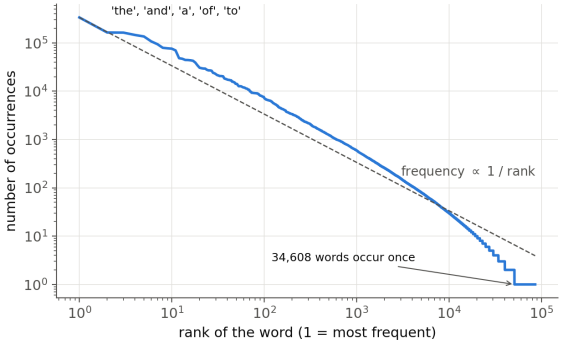
Word	Porter stem	WordNet lemma
studies	studi	study
studying	studi	study (as a verb)
ran	ran	run (as a verb)
better	better	good (as an adjective)
wolves	wolv	wolf
organization	organ	organization
university	univers	university
universe	univers	universe

NLTK 3.9.1 `PorterStemmer` and `WordNetLemmatizer`, run for this slide.

- A **stemmer** chops suffixes by rules. Fast and dictionary-free; its output need not be a word (*studi*).
- It can **over-stem**: *organization* becomes *organ*, and *university* and *universe* collapse into one feature.
- A **lemmatizer** looks words up in a dictionary and returns a real word. It needs the part of speech: *better* becomes *good* only when tagged as an adjective.
- For Arabic, the ISRI stemmer maps المكتبة, مكتوب, يكتبون and كاتب all to the root كتب, but leaves فسيكتبونها as فسيكتبون.

Porter, "An algorithm for suffix stripping," *Program* 14(3), 1980, doi:10.1108/eb046814.

Word Frequencies Follow Zipf's Law



Word counts in the 25,000 IMDb training reviews (Maas et al., 2011), lower-cased, letters only; computed for this course.

- Sort words by frequency: the count falls roughly as $1/\text{rank}$, a straight line on log-log axes (**Zipf's law**).
- In these reviews, 5.8 million tokens use 85,680 distinct words. The 100 most frequent words make up half of all tokens.
- At the other end, 34,608 words (40% of the vocabulary) occur exactly once.
- Both ends cause trouble: frequent words dominate raw counts, and rare words get too little data to learn from. TF-IDF addresses the first; subword tokenization, the second.

How Much Preprocessing Today?

Step	Count-based models, keyword search	Pretrained neural models (subword tokenizers)
Remove markup, URLs	yes	yes
Unicode NFKC	yes	usually; the tokenizer may already apply it
Lower case	usually	no: the model was trained on cased text
Stop-word removal	for topic tasks only	no: function words carry grammar and negation
Stemming, lemmatization	often helps	no: the tokenizer splits rare words itself
Arabic diacritics, alef forms	normalise	keep raw text, unless the model card says otherwise

- The rule: preprocess the way the model's training data was preprocessed. A pretrained model sees raw text, so feed it raw text.
- The classic pipeline remains the right choice for TF-IDF baselines and keyword search, which are still part of production retrieval systems.

The Arabic Script in One Slide

- The pipeline was designed for English. Arabic breaks several of its assumptions, starting with the script.
- 28 letters, written **right to left**. Most letters join their neighbours and change shape with position (start, middle, end of a word, or alone).
- No upper or lower case: no capital letter marks a name or the start of a sentence.
- Short vowels are **diacritics** written above or below the letters. Everyday text (news, the web, social media) usually leaves them out; the reader restores them from context.
- Text on screen is a sequence of Unicode code points in logical order; shaping and right-to-left display are done by the renderer, so a program sees the letters in reading order.

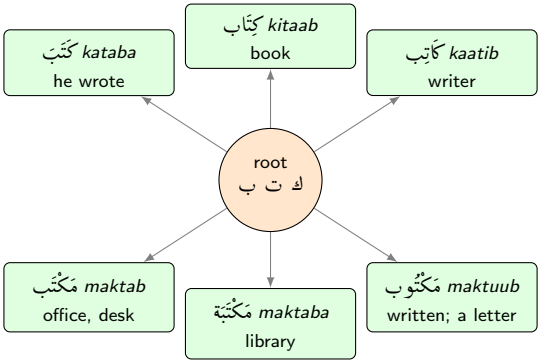
Form	Text
letters alone	ك ت ا ب
joined word	كتاب
with diacritics	كِتَابٌ
meaning	"a book"

The joined and the diacritised forms are the same word. A tokenizer that keeps diacritics sees two different strings.

N. Y. Habash, *Introduction to Arabic Natural Language Processing*, Morgan & Claypool (Synthesis Lectures on Human Language Technologies), 2010, doi:10.1007/978-3-031-02139-8.

Root-and-Pattern Morphology

- Most Arabic words are built from a consonantal **root** that carries a core meaning, poured into a **pattern** of vowels and extra letters that sets the word type.
- The root k-t-b (ك ت ب) is about writing. Six of its words are on the right.
- Related words therefore share letters that are *not* contiguous: *maktaba* and *kitaab* have no common prefix or suffix, so suffix-stripping stemmers built for English cannot connect them.



One root, six words: the pattern adds vowels and letters around the same three consonants.

One Written Word Can Be a Whole Sentence

- Conjunctions, prepositions, the article, future markers and object pronouns attach to the word as **clitics**, with no space between them.

Written word	Pieces, in reading order				English
فسيكتبونها	ف *fa* so	س *sa* will	يكتبون *yaktubuuna* they write	ها *haa* it	"so they will write it"
وبالكتاب	و *wa* and	ب *bi* with	ال *al* the	كتاب *kitaab* book	"and with the book"

- Splitting on spaces gives one "word" where English has four or five. Every combination of clitics becomes its own vocabulary entry, so the vocabulary grows quickly and most entries are rare.
- Arabic tools therefore **segment** clitics before counting words, and subword tokenizers learn similar splits from data.

Missing Diacritics Create Ambiguity

Diacritised	Reading	Meaning
عِلْم	*ilm*	knowledge, science
عَلَم	*alam*	flag
عَلَّمَ	*allama*	he taught
عَلِمَ	*alima*	he knew
عُلِمَ	*ulima*	it was known

All five are written علم once the diacritics are dropped.

- A program that sees only علم must choose between a noun and three verb forms from the surrounding words.
- Partial diacritics make it worse: the same word can appear bare, partly marked and fully marked, as three strings.
- Common practice for counting and search is to **remove** all diacritics so the forms merge; tasks that need the reading, such as speech synthesis, need a **diacritizer** that restores them.

Dialects, Code-Switching and Arabizi

- **Modern Standard Arabic (MSA)** is the written standard of news and formal text. People speak, and increasingly write online, in regional **dialects** that differ in words, pronunciation and grammar.
- The MADAR project built a parallel corpus of **25 Arabic city dialects**: the same travel-domain sentences written in each.
- **Code-switching** mixes Arabic with English or French in one sentence. **Arabizi** writes Arabic in Latin letters, using digits for sounds Latin lacks: *7abibi* for حبيبي (7 stands for ح) and *3arabi* for عربي (3 stands for ع).
- A model trained only on MSA sees many dialect words for the first time.

"What do you want?"	
MSA	ماذا تريد؟
Egyptian	عايز إيه؟
Gulf (Saudi)	وش تبي؟
Levantine	شو بدك؟
Moroccan	آش بغيتي؟

Five varieties, almost no shared words.

Bouamor et al., "The MADAR Arabic Dialect Corpus and Lexicon," LREC 2018, L18-1535.

Normalising and Segmenting Arabic

Rule	Before	After
unify alef forms	أ إ آ	ا
taa marbuta to haa	مدرسة	مدرسه
alef maqsura to yaa	على	علي
remove tatweel (stretching)	كتـــاب	كتاب
remove diacritics	كِتَابٌ	كتاب

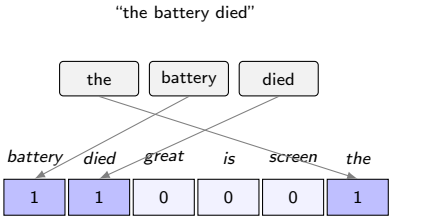
Normalisation merges spelling variants, and sometimes merges distinct words too: the preposition على (*on*) and the name علي (*Ali*) become one string.

- **Root stemmers** such as ISRI (in NLTK) strip affixes and return the root, without a root dictionary; long clitic chains defeat them.
- **Segmenters** such as Farasa split a word into its clitics and stem, marking each boundary with a plus sign.
- **CAMeL Tools** (Python) covers pre-processing and normalisation, morphological analysis, dialect identification, named entity recognition and sentiment analysis.
- For transformer models, keep the raw text and let the subword tokenizer split it; apply these rules for keyword search and count-based features.

Abdelali et al., "Farasa," NAACL 2016 Demos, N16-3003; Obeid et al., "CAMeL Tools," LREC 2020, 2020.lrec-1.868.

Models Need a Fixed-Length Vector

- After preprocessing, a document is a list of tokens of any length: 3 words or 2,000. A classifier needs the same number of inputs for every example.
- **Bag of words:** fix a list of words, the **vocabulary** of V words, and describe every document by one number per vocabulary word, how often that word occurs in it.
- Every document now has V numbers, whatever its length.
- Word order is thrown away; only which words occur, and how often, is kept. Hence "bag".



one count per vocabulary word: a vector of length $V = 6$

Step 1: Build the Vocabulary

- Collect every distinct token of the **training** documents, after preprocessing, and give each one a column number. The order is a convention; any fixed order works.
- On the 25,000 IMDb training reviews, scikit-learn's default tokenizer finds 74,849 distinct tokens. Keeping only words that occur in at least two reviews leaves 44,797: a word seen in one review gives too little evidence to learn anything about it.
- The vocabulary is frozen after training. A word that first appears at test time has no column and is simply ignored.

d_1: "the battery is great"
d_2: "the screen is great"
d_3: "the battery died"

column	word
0	battery
1	died
2	great
3	is
4	screen
5	the

Six distinct words, sorted alphabetically. IMDb counts computed for this course (lower-cased, `<br />` removed).

Step 2: Fill the Document-Term Matrix

- One row per document. Go through its tokens and add 1 to the column of each one.
- d_1 = "the battery is great": 1 under *battery*, *great*, *is* and *the*; 0 under *died* and *screen*.
- A repeated word counts more than once: "great, really great" puts 2 under *great*. A **binary** variant records only presence, 1 or 0.
- Read by **rows**, the matrix gives one vector per document; read by **columns**, it shows in which documents each word occurs.

	battery	died	great	is	screen	the
d_1	**1**	0	**1**	**1**	0	**1**
d_2	0	0	1	1	1	1
d_3	1	1	0	0	0	1

The document-term count matrix: one row per document, one column per vocabulary word.

What the Vector Keeps, and What It Loses

Keeps

- Which words occur and how often: enough to tell a review about batteries from one about screens, or praise from complaint.
- Documents about the same thing share columns. d_1 and d_2 share 3 words (*great*, *is*, *the*), d_1 and d_3 share 2, d_2 and d_3 only *the*.
- Counting shared words is the dot product of two rows: 3, 2 and 1 here.

Loses

- **Order:** "the dog bit the man" and "the man bit the dog" get the same row.
- **Scope:** "not good, bad" and "good, not bad" get the same row too.
- **Similarity of words:** *great* and *excellent* are separate columns, as unrelated as *great* and *battery*.
- **Balance:** the most frequent words, such as *the*, get the largest counts and say the least.

For count-based models, n-gram features bring back some order and TF-IDF reweights the counts. Word similarity needs learned word vectors.

Real Bag-of-Words Vectors Are Sparse

IMDb, 25,000 training reviews	
columns (vocabulary)	44,797
distinct words per review, mean	136
distinct words per review, median	111
non-zero entries	0.30%
stored densely (32-bit numbers)	4.5 GB
stored sparsely	41 MB

Words seen in at least two reviews; scikit-learn `CountVectorizer`. Computed for this course.

- A review uses about a hundred of the 44,797 words, so more than 99.6% of every row is 0.
- **Sparse** storage keeps only the non-zero entries, as (column, count) pairs: about a hundred times less memory here.
- Linear models compute directly on sparse rows, touching only the non-zero entries. That is why count-based classifiers train in seconds on a laptop.

N-gram Features Recover Local Word Order

- Single words lose order. Adding **n-grams** as extra columns puts some of it back: unigrams ($n = 1$), bigrams ($n = 2$), trigrams ($n = 3$).
- With unigrams only, "not good" and "good" share the feature *good*. The bigram *not good* is a separate feature with its own weight.
- The price is size: on the IMDb reviews, adding bigrams (seen at least twice) grows the feature set from 44,797 to 436,597.

"the movie was not good"

unigrams: *the*, *movie*, *was*, *not*, *good*

bigrams: *the movie*, *movie was*, *was not*, **not good**

Features from scikit-learn's `CountVectorizer(ngram_range=(1, 2))`; feature counts measured for this course.

TF-IDF: Weight Words by How Informative They Are

- Raw counts favour words that appear everywhere. A word that occurs in every document cannot tell documents apart.
- Sparck Jones (1972) proposed weighting a term by how few documents contain it. Combined with the count inside the document:

$$\text{tf-idf}(t, d) \;=\; \text{tf}(t, d) \times \log \frac{N}{\text{df}(t)}$$

  where $\text{tf}(t, d)$ is the number of times term t occurs in document d, N is the number of documents, and $\text{df}(t)$ is the number of documents that contain t.
- The second factor is the **inverse document frequency** (IDF). It is 0 for a word in every document and largest for a word in exactly one.
- Libraries add small variations. scikit-learn uses $\ln \frac{1+N}{1+\text{df}(t)} + 1$, so no weight is exactly zero, and then scales each document vector to unit length.

Sparck Jones, "A statistical interpretation of term specificity and its application in retrieval," *Journal of Documentation* 28(1), 1972, doi:10.1108/eb026526.

TF-IDF, Worked by Hand

Same three documents, $N = 3$, logarithm base 10.

Term	**df**	**idf** $= \log_{10}(3/\text{df})$
the	3	0.000
battery, great, is	2	0.176
died, screen	1	0.477

Each term occurs at most once per document, so tf-idf equals idf where the term is present.

	battery	died	great	is	screen	the
d_1	.176	0	.176	.176	0	0
d_2	0	0	.176	.176	**.477**	0
d_3	.176	**.477**	0	0	0	0

- *the* vanishes from every document; *screen* and *died*, each in one document only, now carry the most weight. They are the words that say what d_2 and d_3 are about.
- Every document is now a fixed-length vector of weights in which the informative words stand out: ready for a classifier.

A First Classifier: Which Words Appear in Which Class?

- With every document a vector of word counts, the simplest classifier compares how often each word occurs in each class of the training data.
- Four labelled reviews give the counts on the right. *great* and *love* occur only in positive reviews, *died* only in negative ones; *screen* and *battery* occur in both.
- So a new review containing *died* is probably negative, and one containing *screen* tells us little.
- **Naive Bayes** turns this intuition into probabilities and a decision rule.

Training data, after lower-casing and removing *the*:

pos	great battery, great screen
pos	love screen
neg	battery died fast
neg	screen cracked, battery died

Word	**count in pos**	**count in neg**
great	2	0
love	1	0
screen	2	1
battery	1	2
died	0	2
fast, cracked	0	1 each
tokens	6	7

The bag-of-words counts of each class, added up: $|V| = 7$ distinct words.

Bayes' Rule Turns Counts into a Decision

- We want the class c with the highest $P(c \mid d)$ for a document d. The counts describe the other direction: how typical the words are of each class, $P(d \mid c)$.
- Bayes' rule connects the two:

$$P(c \mid d) \;=\; \frac{P(d \mid c)\,P(c)}{P(d)}$$

- $P(d)$ is the same for every class, so it can be dropped when comparing classes:

$$\hat{c} \;=\; \arg\max_{c} \; P(d \mid c)\,P(c)$$

- $P(c)$ is the **prior**: how common the class is before reading the text. $P(d \mid c)$ is the **likelihood**: how typical the text is of that class.

A generative story

The model pretends each review was written in two steps:

1. pick a class with probability $P(c)$;
2. draw the words one by one from that class's word distribution $P(w \mid c)$.

Classifying a review asks: which class most probably wrote it?

A model of how the data are generated is called **generative**; one that models $P(c \mid d)$ directly, such as logistic regression, is **discriminative**.

Naive Bayes: Treat the Words as Independent

- $P(d \mid c)$ for a whole document cannot be estimated directly: almost every document is unique.
- Two simplifications make it a product of word probabilities:
  - **bag of words:** word positions do not matter;
  - **the naive assumption:** given the class, each word is independent of the others.

$$P(w_1, \ldots, w_n \mid c) \approx \prod_{i=1}^{n} P(w_i \mid c)$$

- Multiplying hundreds of probabilities below 1 underflows to 0 in floating point, so the classifier works with sums of logarithms:

$$\hat{c} = \arg\max_c \left[\log P(c) + \sum_{i=1}^{n} \log P(w_i \mid c) \right]$$

Why "naive"?

Words are not independent: after *San*, the word *Francisco* is far more likely than average, and *not* changes what *good* means.

The assumption is wrong, yet the classifier is often accurate. It only has to rank the classes correctly, not to get the probabilities right.

Training Is Counting, with Smoothing

- Prior: the share of training documents in class c.

$$P(c) = \frac{N_c}{N_{\text{doc}}}$$

- Likelihood: how often w appears in class c, out of all word tokens of that class, with one added to every count:

$$P(w \mid c) = \frac{\text{count}(w, c) + 1}{\sum_{w'} \text{count}(w', c) + |V|}$$

- Without the $+1$ (**Laplace smoothing**), a word never seen in a class gets probability 0, and one such word sets the whole product to 0. The $+|V|$ keeps each class's probabilities summing to 1.

From the four reviews

Two documents per class: $P(\text{pos}) = P(\text{neg}) = 2/4 = 0.5$.

Positive tokens: 6, so the denominator is $6 + 7 = 13$:

$$P(\text{great} \mid \text{pos}) = \frac{2+1}{13} = 0.231$$

$$P(\text{died} \mid \text{pos}) = \frac{0+1}{13} = 0.077$$

Without smoothing the second would be $0/6 = 0$, and any positive review mentioning *died* would get probability 0.

Worked Example: Each Word Casts a Vote

Word	$P(w \mid \text{pos})$	$P(w \mid \text{neg})$	$\lambda(w)$
great	$3/13 = .231$	$1/14 = .071$	$+1.17$
love	$2/13 = .154$	$1/14 = .071$	$+0.77$
screen	$3/13 = .231$	$2/14 = .143$	$+0.48$
battery	$2/13 = .154$	$3/14 = .214$	-0.33
fast, cracked	$1/13 = .077$	$2/14 = .143$	-0.62
died	$1/13 = .077$	$3/14 = .214$	-1.02

Denominators: $6 + 7 = 13$ for pos and $7 + 7 = 14$ for neg. Natural logarithms.

- With two classes, compare them through the **log-likelihood ratio** of each word:

$$\lambda(w) = \log \frac{P(w \mid \text{pos})}{P(w \mid \text{neg})}$$

- $\lambda > 0$ is a vote for positive, $\lambda < 0$ for negative, and $\lambda = 0$ a word that says nothing.
- *died* gets 0 in the positive counts but still a non-zero probability, thanks to the $+1$.

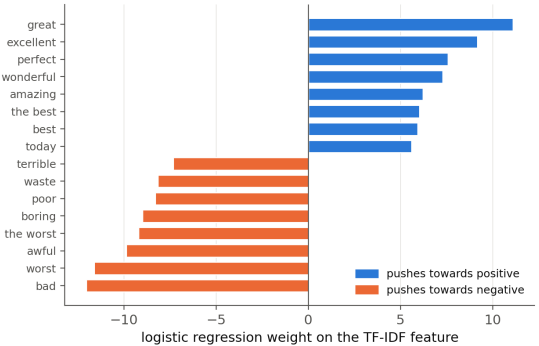
Classifying New Reviews

- Score $= \log \frac{P(\text{pos})}{P(\text{neg})} + \sum_i \lambda(w_i)$. Here the classes are balanced, so the first term is $\log 1 = 0$. Words outside V (*the*, *but*) are skipped.

Review	Votes	Score	Class
"the battery died"	battery -0.33, died -1.02	-1.36	neg
"love it"	love $+0.77$	$+0.77$	pos
"great screen but the battery died"	great $+1.17$, screen $+0.48$, battery -0.33, died -1.02	$+0.30$	pos

- The mixed review comes out barely positive: each word votes on its own, and nothing in the model knows that *but* shifts the weight to what follows.
- With unbalanced classes the prior term matters: if 90% of the training reviews were negative, every score would start at $\log(0.1/0.9) = -2.2$.
- Training is a single pass of counting, and the model is a table of $|V|$ numbers per class, so Naive Bayes trains in seconds on millions of documents.

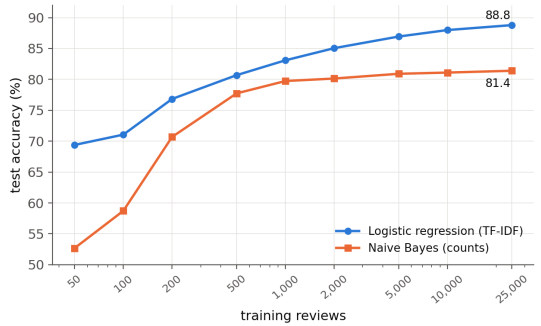
From Counted Votes to Learned Weights



The largest weights of a logistic regression on IMDb TF-IDF unigram and bigram features; computed for this course.

- Naive Bayes fixes each word's vote from counts, one word at a time. **Logistic regression** instead learns one weight per feature by minimising the classification loss on all features together, the discriminative counterpart of $\lambda(w)$. With hundreds of thousands of sparse features it trains in seconds on a CPU, as does a linear support vector machine (SVM).
- The weights are readable: the features that push a review towards each class are the words a person would pick, plus bigrams such as *the worst*.
- Odd entries such as *today* show what the model latched onto in this dataset; reading the weights is a first error check.

Naive Bayes or Logistic Regression?



IMDb test accuracy against the number of training reviews (balanced, mean of 3 random draws; vocabulary from the full training set). Multinomial Naive Bayes on unigram counts; logistic regression on unigram TF-IDF. Computed for this course.

- Ng and Jordan (2002) found "two distinct regimes": logistic regression has the lower error with enough data, but Naive Bayes "may also approach its (higher) asymptotic error much faster".
- On IMDb, logistic regression was ahead at every size. Naive Bayes came within 2 points of its full-data accuracy after 1,000 reviews; logistic regression needed 5,000, and kept climbing to finish 7 points higher.
- The crossover Ng and Jordan describe depends on the data and the features. Both models train in seconds, so run both on your own data and keep the better one.

Ng and Jordan, "On Discriminative vs. Generative Classifiers: A comparison of logistic regression and naive Bayes," NeurIPS 2001 (published 2002), proceedings.neurips.cc.

Why Count-Based Classifiers Are Still the First Baseline

IMDb test accuracy	Unigrams	+ Bigrams
Naive Bayes	81.4%	85.7%
Logistic regression	88.8%	90.5%

25,000 training and 25,000 test reviews, words seen in at least 2 reviews (`min_df=2`). Naive Bayes: counts, add-one smoothing. Logistic regression: TF-IDF, sublinear tf, $C = 4$; 7 s and 16 s on one CPU. Run for this course.

- Wang and Manning (2012): bigram features gave consistent gains on sentiment tasks, and simple Naive Bayes and SVM variants outperformed most published results on sentiment datasets.
- For search, the BEIR benchmark (18 datasets) found BM25, a TF-IDF-style keyword scorer, to be "a robust baseline"; dense neural retrievers "often underperform" other approaches out of domain.

Run it first, every time

- It sets the number any larger model must beat to justify its cost.
- It finds label noise and leakage early, through its readable weights.
- It runs anywhere: no GPU, no model download, no data leaving the building.

Wang and Manning, "Baselines and Bigrams," ACL 2012, P12-2018; Thakur et al., "BEIR," NeurIPS 2021 Datasets and Benchmarks, arXiv:2104.08663v4.

From Classifying Text to Predicting the Next Word

- Bag-of-words classifiers ignore word order, and still work for labelling whole documents. Many tasks need order: a fluent sentence should score higher than the same words scrambled.
- A **language model** assigns a probability to every sequence of words:

$$P(\text{the teacher drinks tea}) > P(\text{tea drinks the teacher})$$

- Equivalently, it gives the probability of each possible **next word** after a history: $P(w \mid \text{the teacher drinks})$.
- Large language models are trained on exactly this next-word objective. The simplest such model needs nothing but counts.

Where next-word probabilities are used

- **Autocomplete:** suggest the most probable next words.
- **Speech recognition:** "recognise speech" and "wreck a nice beach" sound alike; the language model prefers the likelier text.
- **Spelling correction:** "*their* are two" is less probable than "*there* are two".
- **Generation:** sample a word, append it, repeat.

The Probability of a Sentence: the Chain Rule

- Given a sentence, what is its probability?

$$P(the\ teacher\ drinks\ tea) = ?$$

- Conditional probability and chain rule reminder

$$P(B|A) = \frac{P(A,B)}{P(A)} \implies P(A,B) = P(A)P(B|A)$$

$$P(A,B,C,D) = P(A)P(B|A)P(C|A,B)P(D|A,B,C)$$

$$P(the\ teacher\ drinks\ tea) = P(the)P(teacher|the)P(drinks|the\ teacher)P(tea|the\ teacher\ drinks)$$

- Write w_1^n for the words $w_1 \dots w_n$. The chain rule of probability breaks a sentence into one next-word prediction per word, and it is exact: $P(w_1^n) = \prod_{i=1}^{n} P(w_i \mid w_1^{i-1})$.
- Each word is conditioned on **all** the words before it. The last factor, $P(\text{tea} \mid \text{the teacher drinks})$, would be estimated from the counts of "the teacher drinks" and of "the teacher drinks tea".

Figures: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 3: Autocomplete and Language Models.

Long Histories Are Never Seen: the Markov Assumption

Input: the teacher drinks tea

$$P(\textit{the teacher drinks tea}) = P(\textit{the})P(\textit{teacher}|\textit{the})P(\textit{drinks}|\textit{the teacher})P(\textit{tea}|\textit{the teacher drinks})$$

$$P(\textit{tea}|\textit{the teacher drinks}) = \frac{C(\textit{the teacher drinks tea})}{C(\textit{the teacher drinks})}$$ ← Both likely 0

Both counts are probably 0, so the estimate is 0/0.

the teacher drinks tea

$$P(\textit{tea}|\textit{the teacher drinks}) \approx P(\textit{tea}|\textit{drinks})$$

$$P(\textit{teacher}|\textit{the})$$
$$P(\textit{drinks}|\textit{teacher})$$
$$P(\textit{tea}|\textit{drinks})$$

$$P(\textit{the teacher drinks tea}) =$$
$$P(\textit{the})P(\textit{teacher}|\textit{the})P(\textit{drinks}|\textit{the teacher})P(\textit{tea}|\textit{the teacher drinks})$$
$$\Downarrow$$
$$P(\textit{the})P(\textit{teacher}|\textit{the})P(\textit{drinks}|\textit{teacher})P(\textit{tea}|\textit{drinks})$$

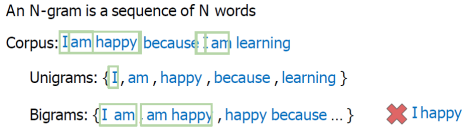
- Markov assumption: only last N words matter
- Bigram $P(w_n|w_1^{n-1}) \approx P(w_n|w_{n-1})$
- N-gram $P(w_n|w_1^{n-1}) \approx P(w_n|w_{n-N+1}^{n-1})$
- Entire sentence modeled with bigram $P(w_1^n) \approx \prod_{i=1}^{n} P(w_i|w_{i-1})$

$$P(w_1^n) \approx P(w_1)P(w_2|w_1)...P(w_n|w_{n-1})$$

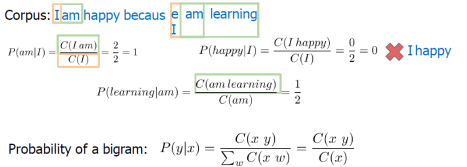
- Most sentences, and most of their long prefixes, appear nowhere in any corpus, so the exact factors cannot be counted.
- **Markov assumption:** only the last $N-1$ words matter. A bigram model keeps one word of history; the sentence probability becomes a product of short factors that can be counted.

Figures: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 3.

Counting Short Histories: N-gram Estimates



The unigrams, bigrams and trigrams of one corpus.

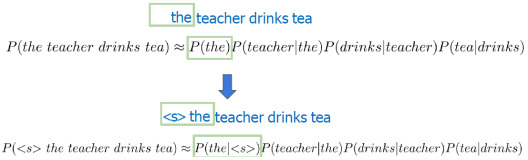


Bigram: count of the pair over the count of its first word.

- The Markov assumption needs the probability of a word given the last $N-1$ words: count the N-gram, divide by the count of its history. For bigrams, $P(y \mid x) = C(x\,y)/C(x)$, where $C(\cdot)$ is a count in the corpus.
- In "I am happy because I am learning": "I" occurs twice and "I am" twice, so $P(\text{am} \mid \text{I}) = 2/2 = 1$; "I happy" never occurs, so $P(\text{happy} \mid \text{I}) = 0$.
- These relative frequencies are the **maximum likelihood estimates**: the values that make the training corpus most probable. A longer history makes predictions sharper and counts smaller.

Figures: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 3.

Marking the Start of a Sentence



- Trigram:

$$P(the\ teacher\ drinks\ tea) \approx P(the)P(teacher|the)P(drinks|the\ teacher)P(tea|teacher\ drinks)$$

the teacher drinks tea => <s> <s> the teacher drinks tea

$$P(w_1^n) \approx P(w_1|<s>\ <s>)P(w_2|<s>\ w_1)...P(w_n|w_{n-2}\ w_{n-1})$$

- N-gram model: add N-1 start tokens <s>

- Counting needs a history for every word, and the first word has none. Its unigram probability P(the) ignores that "the" is a typical *first* word.
- Prepend a start token <s> and use P(the | <s>): how often sentences *begin* with "the".
- An N-gram model needs $N-1$ start tokens, so that every word, including the first, has a full history.

Figures: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 3.

Marking the End of a Sentence

Corpus

<s> yes no

<s> yes yes

<s> no no

Sentences of length 2:

<s> yes yes

<s> yes no

<s> no no

<s> no yes

$$P(< s > \text{ yes yes}) = \frac{1}{3}$$
$$P(< s > \text{ yes no}) = \frac{1}{3}$$
$$P(< s > \text{ no no}) = \frac{1}{3}$$
$$P(< s > \text{ no yes}) = 0$$
$$\sum_{2 \text{ word}} P(\cdots) = 1$$

- Bigram

<s> the teacher drinks tea => <s> the teacher drinks tea </s>

$$P(the|<s>)P(teacher|the)P(drinks|teacher)P(tea|drinks)P(</s>|tea)$$

Corpus:

<s> Lyn drinks chocolate </s>

<s> John drinks </s>

$$\sum_{w} C(drinks\ w) = 2$$
$$C(drinks) = 2$$

Without an end token: corpus of three *yes*/*no* sentences.

- With only start tokens, the four sentences of length 2 already sum to $\frac{1}{3} + \frac{1}{3} + \frac{1}{3} + 0 = 1$, and so do those of length 3, and of every length: the total is far above 1.
- The model has no way to say that a sentence **stops**.

With an end token </s>.

- Append </s> to every training sentence and treat it as an ordinary word: ending gets its own probability, *P*(</s> | tea), and all lengths together sum to 1.
- One </s> is enough for any *N*, because it is predicted, never used as history. Generation stops when it is sampled.

Figures: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 3.

From a Count Matrix to a Probability Matrix

$$P(w_n|w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1}, w_n)}{C(w_{n-N+1}^{n-1})}$$

- Rows: unique corpus (N-1)-grams
- Columns: unique corpus words

Corpus: <s>I study I learn</s>

- Bigram count matrix

"study I" bigram

	<s>	</s>	I	study	learn
<s>	0	0	1	0	0
</s>	0	0	0	0	0
I	0	0	0	1	1
study	0	0	1	0	0
learn	0	1	0	0	0

- Divide each cell by its row sum

Corpus: <s>I study I learn</s>

Count matrix (bigram)

	<s>	</s>	I	study	learn	sum
<s>	0	0	1	0	0	1
</s>	0	0	0	0	0	0
I	0	0	0	1	1	2
study	0	0	1	0	0	1
learn	0	1	0	0	0	1

$$P(w_n|w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1}, w_n)}{C(w_{n-N+1}^{n-1})}$$

$$sum(row) = \sum_{w \in V} C(w_{n-N+1}^{n-1}, w) = C(w_{n-N+1}^{n-1})$$

Probability matrix

	<s>	</s>	I	study	learn
<s>	0	0	1	0	0
</s>	0	0	0	0	0
I	0	0	0	0.5	0.5
study	0	0	1	0	0
learn	0	1	0	0	0

- With start and end tokens in place, count every bigram once. **Count matrix:** one row per history, one column per next word; cell = how often that word followed that history.
- **Probability matrix:** divide each cell by its row sum. Each row is now a probability distribution over the next word. This table is the whole trained model.

Figures: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 3. Corpus: "<s> I study I learn </s>".

Worked Example: Score a Sentence

Corpus

<s Lyn drinks chocolate </s>
<s> John drinks tea </s>
<s> Lyn eats chocolate </s>

$$P(sentence) = \frac{2}{3} * \frac{1}{2} * \frac{1}{2} * \frac{2}{2} = \frac{1}{6}$$

$$P(John|<s>) = \frac{1}{3} \qquad P(</s>|tea) = \frac{1}{1}$$

$$P(chocolate|eats) = \frac{1}{2} \qquad P(Lyn|<s>) = ? = \frac{2}{3}$$

Objective: Apply sequence probability approximation with bigrams after adding start and end word.

Question:
Given these conditional probabilities
P(Mary)=0.1; P(likes)=0.2; P(cats)=0.3
P(Mary|<s>)=0.2; P(</s>|cats)=0.6
P(likes|Mary) =0.3; P(cats|likes)=0.1

Approximate the probability of the following sentence with bigrams: "<s> Mary likes cats </s>"
Type: Multiple Choice, single answer

Options and solution:

1. P(<s> Mary likes cats </s>) = 0
2. P(<s> Mary likes cats </s>) =0.0036
3. P(<s> Mary likes cats </s>) = 0.003
4. P(<s> Mary likes cats </s>) = 1

$P(\text{<s> Lyn drinks chocolate </s>}) = \frac{2}{3} \cdot \frac{1}{2} \cdot \frac{1}{2} \cdot \frac{2}{2} = \frac{1}{6}$

$0.2 \times 0.3 \times 0.1 \times 0.6 = 0.0036$

- Left: estimate each factor by counting in a three-sentence corpus, then multiply along the sentence, from <s> to </s>. Right: the same product from given bigram probabilities. Cover the right-hand answer and compute it first.

Figures: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 3.

Using the Model: Predict, Score, and Add Logs

- probability matrix => language model
  - Sentence probability
  - Next word prediction

	<s>	</s>	I	study	learn
<s>	0	0	1	0	0
</s>	0	0	0	0	0
I	0	0	0	0.5	0.5
study	0	0	1	0	0
learn	0	1	0	0	0

Sentence probability:
<s> I learn </s>

$$P(sentence) = P(I|<s>)P(learn|I)P(</s>|learn) = 1 \times 0.5 \times 1 = 0.5$$

DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 3.

- **Next-word prediction:** read the row of the current history and take, or sample, a likely column.
- **Sentence probability:** multiply the cells along the sentence's path through the matrix.
- **In practice, add log probabilities:** a product of 30 factors around 0.01 is 10^{-60}, and longer texts underflow to 0. Sums of logs do not:

$$\log P(w_1^n) \approx \sum_{i=1}^{n} \log P(w_i \mid w_{i-1})$$

A Bigram Model of Movie Reviews

$P(w \mid$ <s>)	
the	.121
i	.092
this	.048
it	.040
but	.025
and	.024

$P(w \mid$ movie)	
.	.167
is	.108
,	.090
was	.047
that	.027
and	.026

- The same recipe on real text: two rows of the probability matrix of the 25,000 IMDb training reviews, split into 281,317 sentences (5.1 million tokens, punctuation included).
- "movie" occurs 37,581 times; it is followed by a full stop 6,263 times, so $P(. \mid \text{movie}) = 6263/37581 = .167$.

- Each row is a distribution over every word that ever followed the history; the six largest cells are shown. Most cells of the full matrix are 0.

Lower-cased words and punctuation as tokens; reviews split into sentences at ".", "!" and "?". Computed for this course.

Generating Text by Sampling

- Start from <s>, sample the next word from its row, append it, and repeat until </s>. Models trained on the same IMDb sentences:

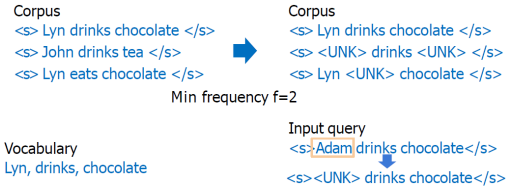
N	**Sampled sentence**
1	call vin acting some competitor the clearly , . was , disappointing . been way is or the the
2	i will hit the script , composing and back good , gaots seems to be seen ?
3	she marries one round , she'll date and have enjoyed all of the lame finish where the directing doesn't get done laughting at humanity i weep .
4	ya see , this film does not win all at this years wrestlemania's match card , i was a little bit better .

- Unigrams give word salad; bigrams give locally plausible pairs; trigrams give phrases that read well for a few words, then drift.
- With 4-grams, long runs come straight from the training text: "at this years wrestlemania's match card , i was" (9 tokens) appears word for word in the training reviews. A longer history reads better but copies more and generalises less.

Spellings such as *gaots* and *laughting* are copied from the reviews. One sample per model, fixed random seed; computed for this course.

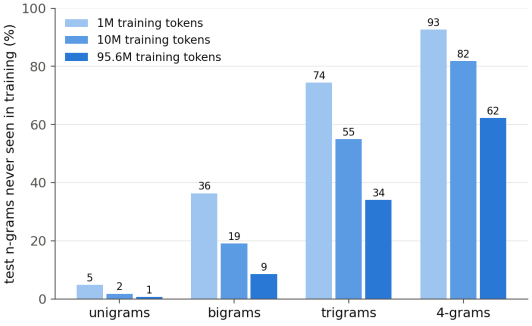
Words the Model Has Never Seen: <UNK>

- A test sentence can contain a word with no row and no column in the matrix. A **closed vocabulary** model only knows the words of a fixed list V; any other word is **out of vocabulary** (OOV).
- Recipe: choose V (for example, words seen at least f times), replace every other word in the training corpus by the token <UNK>, and count <UNK> like any word. At test time, map unknown words to <UNK> too.
- Keep the <UNK> rate low. Predicting one frequent token is easy, so a large rate makes a model look better than it is, and two models are only comparable with the same V.



With a minimum frequency of 2, only *Lyn*, *drinks* and *chocolate* stay in V. DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 3.

N-grams the Model Has Never Seen: Sparsity



Share of the n-grams in the WikiText-103 test articles that never occur in the first 1, 10 or 95.6 million training tokens (lower-cased words and punctuation). Computed for this course.

- Known words can still form a new combination. Even after 95.6 million training tokens, 34% of the test trigrams and 62% of the test 4-grams never occur in training.
- Ten times more text closes only part of the gap: unseen trigrams fall from 74% to 55% to 34%, while the table grows to 39.5 million distinct trigrams.
- Every unseen n-gram gets probability 0 from counting, and one zero factor sets the whole sentence probability to 0. Smoothing methods move a little probability to unseen n-grams, but cannot tell which unseen ones are plausible.

WikiText-103: Merity et al., "Pointer Sentinel Mixture Models," 2016, arXiv:1609.07843v1.

What Counting Cannot See

Blind spot	Example	Effect
No similarity	seeing "the cat sat" says nothing about "the dog sat"	each word, and each n-gram, is a separate count or column
Synonyms	"car" and "automobile"	different columns, no shared count
Word senses	"bank" of a river and of money	one column and one row for both
Short memory	"I expected it to be great, and it was not"	a bigram model sees one word back
Size	every extra word of history	many more distinct n-grams to store, each seen fewer times

- Bag-of-words classifiers and n-gram models share these blind spots, because both treat every word and n-gram as an unrelated column.
- Bengio et al. (2003) named the problem the curse of dimensionality: a test sequence "is likely to be different from all the word sequences seen during training". Their answer was to learn a dense vector for each word, so that each training sentence informs the model about many similar sentences.

Bengio, Ducharme, Vincent and Jauvin, "A Neural Probabilistic Language Model," *JMLR* 3:1137–1155, 2003, jmlr.org/papers/v3/bengio03a.

Summary

- **Name the output shape first.** One label (classification), one label per token (NER with BIO tags), a span (extractive QA), or new text (summarization, translation, generation). Text-to-text casts all of them as text in, text out.
- **The shape fixes the metric.** Macro-F1 for skewed classes, entity-level F1, exact match and token F1, ROUGE, BLEU and chrF. N-gram metrics reward surface overlap and penalise correct paraphrases.
- **Preprocessing is a set of choices.** Cleaning, NFKC, case folding, tokenization, stop words, stemming and lemmatization shrink the vocabulary but discard information; NLTK's stop list deletes *not*. Feed pretrained models raw text.
- **Arabic adds four problems:** root-and-pattern morphology, attached clitics (one written word can be a sentence), missing diacritics (one bare spelling, five readings), and dialects that share few words.
- **Bag of words** gives every document a fixed-length count vector over a frozen vocabulary: mostly zeros (0.30% non-zero on IMDb), with word order lost. **TF-IDF** $= \mathrm{tf}(t, d) \cdot \log(N/\mathrm{df}(t))$ reweights the counts; words found everywhere get weight 0.
- **Naive Bayes is training by counting:** $\log P(c) + \sum_i \log P(w_i \mid c)$ with add-one smoothing; each word votes with its log-likelihood ratio. It stops improving early; logistic regression keeps using extra data, and on IMDb reaches 90.5% with TF-IDF in seconds on a CPU: the baseline to beat.
- **An n-gram language model** scores a sentence with the chain rule cut to the last $N - 1$ words, using counts: $P(w_i \mid w_{i-1}) = C(w_{i-1}w_i)/C(w_{i-1})$. <s> and </s> make the probabilities of all sentences sum to 1; <UNK> stands for words outside the vocabulary; sampling from the rows generates text.
- **Counting cannot generalise:** most test trigrams never occur in training, synonyms share nothing, and a bigram sees one word back. Dense learned word vectors are the next step.

References

1. Rajpurkar et al., "SQuAD: 100,000+ Questions for Machine Comprehension of Text," EMNLP 2016. arXiv:1606.05250v3
2. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," JMLR 2020. arXiv:1910.10683v4 (source of Figure 1)
3. Papineni et al., "BLEU: a Method for Automatic Evaluation of Machine Translation," ACL 2002. aclanthology.org/P02-1040
4. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," Text Summarization Branches Out, 2004. aclanthology.org/W04-1013
5. Popović, "chrF: character n-gram F-score for automatic MT evaluation," WMT 2015. aclanthology.org/W15-3049
6. Porter, "An algorithm for suffix stripping," *Program* 14(3):130–137, 1980. doi:10.1108/eb046814
7. Bird, Klein and Loper, NLTK 3.9 documentation (tokenizers, Porter and ISRI stemmers, WordNet lemmatizer). nltk.org
8. Habash, *Introduction to Arabic Natural Language Processing*, Synthesis Lectures on Human Language Technologies, 2010. doi:10.1007/978-3-031-02139-8
9. Bouamor et al., "The MADAR Arabic Dialect Corpus and Lexicon," LREC 2018. aclanthology.org/L18-1535
10. Abdelali et al., "Farasa: A Fast and Furious Segmenter for Arabic," NAACL 2016 Demonstrations. aclanthology.org/N16-3003
11. Obeid et al., "CAMeL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing," LREC 2020. aclanthology.org/2020.lrec-1.868

References (cont.)

12. Sparck Jones, “A statistical interpretation of term specificity and its application in retrieval,” *Journal of Documentation* 28(1):11–21, 1972. doi:10.1108/eb026526

13. Maas et al., “Learning Word Vectors for Sentiment Analysis,” ACL 2011 (the IMDb review dataset). aclanthology.org/P11-1015

14. Ng and Jordan, “On Discriminative vs. Generative Classifiers: A comparison of logistic regression and naive Bayes,” NeurIPS 2001. proceedings.neurips.cc

15. Wang and Manning, “Baselines and Bigrams: Simple, Good Sentiment and Topic Classification,” ACL 2012. aclanthology.org/P12-2018

16. Thakur et al., “BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models,” NeurIPS 2021 Datasets and Benchmarks. arXiv:2104.08663v4

17. Jurafsky and Martin, *Speech and Language Processing*, 3rd ed. draft, release of August 19, 2026: Chapter 3, N-gram Language Models; Appendix B, Naive Bayes Classification. web.stanford.edu/~jurafsky/slp3

18. DeepLearning.AI, Natural Language Processing Specialization, Course 2: *NLP with Probabilistic Models*, Week 3: Autocomplete and Language Models (source of the n-gram figures). deeplearning.ai

19. Bengio, Ducharme, Vincent and Jauvin, “A Neural Probabilistic Language Model,” *JMLR* 3:1137–1155, 2003. jmlr.org/papers/v3/bengio03a

20. Merity, Xiong, Bradbury and Socher, “Pointer Sentinel Mixture Models” (the WikiText-103 corpus), 2016. arXiv:1609.07843v1